NEXUS

Manual técnico y preparación para el examen H3

Nombre: Castillo Villarroel Rodrigo

Alcance del documento

Este manual describe la estructura y los conceptos que pueden identificarse en el código compartido de NEXUS. Los conceptos no encontrados como implementación se identifican expresamente y no se presentan como funcionalidades existentes.

1. Arquitectura y funcionamiento general

1.1 ¿Qué es NEXUS?

NEXUS es un programa de consola escrito en C# que simula la exploración de realidades. El usuario registra los datos de un cadete y luego puede explorar, analizar anomalías, revisar el diagnóstico del enlace, recuperar energía, estabilizar el enlace, consultar el inventario y revisar las misiones.

1.2 Inicio y flujo del programa

El punto de entrada está en Program.Main(). El método configura la consola, muestra la presentación, solicita nombre, edad, realidad, energía y estabilidad, crea un objeto Cadete, valida sus condiciones, crea un objeto Nexus, muestra el briefing y llama a MenuPrincipal(nexus). El menú se repite mientras la opción no sea 8; la opción 8 desconecta el sistema.

1.3 Responsabilidad de las clases

Program: inicia el programa y administra el menú. Nexus: coordina el estado principal del juego, el cadete, el inventario, las anomalías y las misiones. Personaje: clase abstracta con datos y comportamientos comunes. Cadete: representa al jugador. Robot y Enemigo: representan entidades con comportamientos propios. Objeto: representa objetos del juego. Inventario: guarda y administra objetos. Mision y Objetivo: representan misiones y sus objetivos. Zona: organiza zonas de exploración. Encuentros: administra los encuentros y combates. Ui: reúne funciones para mostrar textos, títulos, colores y pausas. Entidades: ofrece métodos para crear entidades.

1.4 Programación estructurada y POO

El proyecto combina ambos enfoques. Usa instrucciones, condiciones, ciclos y métodos para controlar el flujo; también usa clases y objetos para organizar datos y comportamientos. Por ejemplo, Program administra el menú, mientras Cadete e Inventario reúnen responsabilidades específicas.

2. Clases, objetos y encapsulamiento

2.1 Clase, objeto e instancia
Una clase define datos y comportamientos; un objeto es una instancia concreta de esa clase. En Main se crea un cadete con: Cadete cadete = new Cadete(nombre, edad, realidad, energia, estabilidad);. Cadete es el tipo, cadete es la variable que referencia el objeto y new crea la instancia.

2.2 Propiedades y métodos
Las propiedades representan información del objeto y los métodos representan acciones. En Cadete aparecen propiedades como Edad, Realidad, Estabilidad y Xp. Sus métodos administran acciones del cadete, como recibir daño, ganar experiencia y mostrar su ficha.

2.3 Constructor y base
El constructor de Cadete recibe nombre, edad, realidad, energía y estabilidad. La parte : base(nombre, energia, 100) llama al constructor de Personaje para inicializar los datos comunes. Después se asignan propiedades propias de Cadete.

2.4 Encapsulamiento y acceso
public permite acceder desde otras clases; private limita el acceso a la propia clase; protected permite el acceso desde la clase y sus derivadas; internal permite el acceso dentro del ensamblado. En el código, public double Estabilidad { get; private set; } permite consultar la estabilidad, pero restringe su modificación directa desde fuera de Cadete.

2.5 this y base
this se refiere al objeto actual. base permite acceder al constructor o a miembros de la clase base. En el constructor de Cadete se usa base(...) para inicializar la parte heredada de Personaje.

2.6 Sobrecarga
La sobrecarga consiste en declarar métodos o constructores con el mismo nombre y distintos parámetros. No se documenta aquí una sobrecarga concreta porque no se ha confirmado una segunda firma del mismo constructor en el código revisado.

3. Herencia, polimorfismo y abstracción

3.1 Herencia
Cadete, Robot y Enemigo derivan de Personaje. Así comparten características generales, pero cada clase puede incorporar propiedades y comportamientos propios.

3.2 Clase abstracta
Personaje se declara abstracta, por lo que no se instancia directamente. Define elementos comunes y declara métodos abstractos que las clases derivadas deben implementar.

3.3 virtual y override
virtual permite que una clase derivada cambie un comportamiento; override indica que se implementa una versión específica heredada. En el proyecto, las clases derivadas sobrescriben métodos como Tipo, Turno, RecibirDano y MostrarFicha.

3.4 Polimorfismo en los encuentros
El sistema de encuentros trabaja con entidades del tipo general Personaje y llama a operaciones como entidad.Turno(cadete). El comportamiento que se ejecuta depende de la clase real del objeto, por ejemplo Robot o Enemigo.

3.5 Interfaces y composición
En el código revisado no se encontró una declaración explícita de interface; por eso no se presenta una interfaz como parte implementada del proyecto. Como concepto, una interfaz declara operaciones que una clase debe implementar. La composición consiste en que una clase use objetos de otras clases; Nexus coordina objetos como el Cadete y el Inventario. Herencia expresa una relación “es un”; composición, una relación “tiene un”.

4. List<T>, recorridos y LINQ

4.1 List<T>
List<T> almacena varios elementos del mismo tipo. Inventario declara una lista privada de objetos: private List<Objeto> objetos;. Esto permite agregar, buscar y eliminar objetos sin exponer directamente la lista interna.

4.2 Recorridos
foreach permite recorrer los elementos de una colección uno por uno. En el proyecto se usa para revisar elementos en distintas operaciones.

4.3 LINQ utilizado en el código
En LeerFrames se obtiene una lista de archivos .txt y se aplican OrderBy(x => x), Take(maximo) y ToArray(). OrderBy ordena los elementos; Take limita la cantidad; ToArray convierte el resultado en un arreglo. La expresión x => x es una lambda que devuelve el propio elemento para usarlo como criterio de ordenamiento.

4.4 Alcance comprobado
En la búsqueda realizada del código se confirmó el uso de OrderBy, Take y ToArray. No se confirmó uso de Where o Select, así que no se describen como operaciones implementadas en NEXUS.

5. Análisis de ejecución y diagramas

5.1 Flujo de inicio
Inicio → configuración de consola → presentación → solicitar datos del cadete → crear Cadete → validar condiciones → crear Nexus → mostrar briefing → abrir menú principal → procesar opción → revisar estados → repetir menú o desconectarse.

5.2 Flujo del menú
MenuPrincipal mantiene un ciclo while mientras la opción sea distinta de 8. Primero muestra el menú y valida que la entrada sea un entero entre 1 y 8. Después ejecuta la acción correspondiente. La opción 1 abre la exploración; 2 analiza anomalías; 3 muestra diagnóstico; 4 recupera energía; 5 estabiliza el enlace; 6 muestra inventario; 7 muestra misiones; 8 desconecta.

5.3 Seguimiento de estado
Para analizar una ejecución, se debe anotar el estado del objeto antes de una acción, identificar el método ejecutado y volver a revisar las propiedades modificadas. Por ejemplo, para una recuperación de energía, el resultado exacto debe determinarse leyendo el método RecargarEnergia y las condiciones de Cadete; no se debe asumir una cantidad de recuperación sin comprobar el código.

5.4 Modificación razonada
Si se agrega una opción al menú, se debe actualizar el texto de MostrarMenu, la validación del rango en MenuPrincipal y la rama que ejecuta la nueva acción. Si solo se agrega el texto, la opción puede seguir siendo rechazada por la validación.

6. Preguntas de preparación para el examen H3

Pregunta 1. ¿Qué diferencia hay entre clase y objeto?
Una clase define datos y comportamientos; un objeto es una instancia creada a partir de esa definición. En NEXUS, Cadete es la clase y la variable cadete recibe una instancia creada con new.

Pregunta 2. ¿Para qué sirve la herencia en NEXUS?
Permite que Cadete, Robot y Enemigo compartan características de Personaje y mantengan sus comportamientos específicos.

Pregunta 3. ¿Qué es el polimorfismo en este proyecto?
Es que una referencia del tipo Personaje pueda ejecutar la versión del método correspondiente a la clase real del objeto, como ocurre al llamar entidad.Turno(cadete).

Pregunta 4. ¿Qué logra el encapsulamiento?
Controla el acceso a los datos. Por ejemplo, Estabilidad permite lectura pública, pero su setter es privado.

Pregunta 5. ¿Para qué sirve List<Objeto> en Inventario?
Permite almacenar y administrar una colección de objetos. La clase Inventario ofrece operaciones para trabajar con esos elementos.

Pregunta 6. ¿Qué hacen OrderBy, Take y ToArray?
OrderBy ordena, Take limita la cantidad de elementos y ToArray convierte el resultado en un arreglo. En NEXUS se aplican a los archivos de animación.

Pregunta 7. ¿Qué diferencia hay entre virtual y override?
Virtual permite redefinir un comportamiento en una clase derivada; override proporciona esa implementación específica.

Pregunta 8. ¿Qué se debe cambiar al agregar una opción al menú?
Se debe actualizar la lista mostrada, la validación de las opciones aceptadas y la lógica que ejecuta la nueva acción.

7. Diagnóstico de aprendizaje

Pendiente de evidencia personal
Esta sección no se completa con respuestas inventadas. La consigna de la actividad indica que el diagnóstico debe basarse en evidencias del trabajo del estudiante, sus explicaciones y ejercicios. Para completarla correctamente, hay que aplicar el PROMPT ESPEJO indicado en la tarea y usar sus resultados reales. Hasta contar con esa evidencia, el diagnóstico queda pendiente.

Referencias al código revisado
Archivo fuente compartido: código C# del proyecto NEXUS Training, namespace NexusTraining. Los nombres de clases, métodos y propiedades se mantienen como aparecen en el código revisado.

PROMPT ESPEJO: DIAGNÓSTICO DE MI APRENDIZAJE

Misión NEXUS: De código a conocimiento

1. Fortalezas demostradas

Durante el desarrollo de NEXUS fui aprendiendo a trabajar con un proyecto de programación más completo, en el que diferentes funcionalidades se conectan para formar un sistema. Participé en las decisiones sobre las características que quería incorporar, como la exploración de realidades, el análisis de anomalías, los mapas, los menús y la administración de la energía y la estabilidad del cadete.

Una de mis fortalezas fue buscar soluciones cuando aparecieron errores. En lugar de abandonar el proyecto, compartí los problemas que encontraba, probé las correcciones y continué trabajando hasta avanzar con las funcionalidades. También fui identificando qué necesitaba cambiar para que el código se adaptara a mi proyecto.

En el código trabajado se utilizan clases y objetos, listas genéricas, recorridos y operaciones de LINQ. Esto me permitió acercarme a conceptos de programación orientada a objetos mediante un ejemplo práctico, en lugar de estudiarlos únicamente de forma teórica.

Considero que mi principal avance fue aprender a trabajar sobre un proyecto real, comprender progresivamente su organización y reconocer que cada clase y método tiene una responsabilidad dentro del sistema. Aunque todavía debo reforzar algunos conceptos, ya tengo una base práctica sobre la que puedo seguir aprendiendo.

2. Aspectos que debo reforzar

Una de las cosas que debo mejorar es explicar con mis propias palabras por qué funciona una parte del código y no limitarme a comprobar que el resultado sea correcto. Durante el proyecto recibí ayuda para resolver algunos problemas, por lo que necesito practicar más la identificación y corrección de errores de forma independiente.

También debo reforzar la explicación de los conceptos de programación orientada a objetos. Por ejemplo, necesito distinguir con seguridad una clase de un objeto, explicar el papel de los constructores y comprender cómo se controla el acceso a los datos mediante el encapsulamiento.

En temas como herencia, polimorfismo e interfaces, debo practicar con ejemplos pequeños para reconocer cómo funcionan y cuándo conviene utilizar cada concepto. En cuanto a las listas y LINQ, necesito avanzar desde reconocer su uso en el código hasta poder construir y explicar consultas sencillas por mi cuenta.

Estas áreas no significan que no haya aprendido nada sobre ellas. Más bien, representan los conocimientos que necesito consolidar para poder demostrar mi aprendizaje con mayor seguridad durante el examen.

3. Diagnóstico del Hito 3

Programación orientada a objetos (POO): Comprensión parcial.

He trabajado con un proyecto organizado mediante clases y objetos, y he participado en la construcción de funcionalidades relacionadas con ellos. Esto me ha permitido conocer su utilidad práctica. Debo reforzar la explicación de los principios de la POO y la diferencia entre este enfoque y la programación estructurada.

Clases y objetos: Comprensión parcial.

En el código aparecen instancias como Cadete y Nexus, que permiten identificar cómo se crean objetos a partir de clases. He trabajado con esta estructura dentro del proyecto, pero necesito practicar la explicación de sus diferencias y de cómo se utilizan los objetos durante la ejecución.

Constructores: Comprensión parcial.

El código utiliza llamadas a constructores para crear objetos e inicializar sus datos. Esto me permite reconocer su función dentro del programa. Debo practicar cómo declarar constructores y explicar cómo se utilizan sus parámetros. No hay suficiente evidencia para confirmar que domino la sobrecarga de constructores.

Encapsulamiento: Comprensión parcial.

En el código aparece, por ejemplo, una propiedad con private set, que limita la modificación directa de su valor desde fuera de la clase. Este caso me permite estudiar cómo se controla el acceso a los datos. Necesito reforzar las diferencias entre los modificadores de acceso y explicar por qué es importante proteger el estado de un objeto.

Herencia: Evidencia insuficiente para confirmar el dominio.

El proyecto contempla diferentes tipos de entidades, pero los fragmentos revisados no permiten confirmar completamente todas las relaciones entre las clases. Debo practicar cómo una clase derivada puede reutilizar y especializar el comportamiento de una clase base.

Polimorfismo: Evidencia insuficiente para confirmar el dominio.

En el código se realizan comprobaciones de tipos de entidades, pero eso no demuestra por sí solo el uso de polimorfismo mediante métodos sobrescritos. Necesito practicar con virtual y override para entender cómo un mismo método puede producir resultados diferentes según el tipo real del objeto.

Interfaces: Evidencia insuficiente para confirmar el dominio.

No hay suficiente evidencia en los fragmentos revisados para confirmar que he implementado o explicado interfaces por mi cuenta. Debo aprender a declarar una interfaz,

implementarla en diferentes clases y comprender cómo permite establecer un contrato común.

Listas genéricas (List<T>): Comprensión parcial.

El proyecto utiliza listas como List<Objeto> y List<string[]>, además de recorridos mediante foreach. Esto demuestra que he trabajado con colecciones dentro del código. Necesito reforzar cómo agregar, eliminar, recorrer y consultar elementos, y explicar qué significa el tipo genérico.

LINQ: Comprensión parcial.

En el código de animaciones se utilizan OrderBy, Take y ToArray para ordenar elementos, limitar los resultados y convertirlos en un arreglo. Este es un ejemplo práctico de LINQ que puedo utilizar para estudiar. Debo practicar cómo escribir consultas propias y comprender operaciones como Where y Select, cuyo uso no se confirmó en los fragmentos revisados.

Balance del Hito 3: tengo una base práctica de trabajo con clases, objetos, listas y operaciones de LINQ. Mi siguiente objetivo es consolidar esos conocimientos y practicar los temas que cuentan con menos evidencia, especialmente herencia, polimorfismo e interfaces. Así podré explicar no solamente qué hace el código, sino también por qué funciona.

4. Mi forma de aprender

Mi forma de aprender durante la misión fue principalmente práctica. Trabajé con NEXUS, propuse funcionalidades, compartí partes del código y busqué soluciones cuando aparecieron errores. También fui indicando qué resultados quería conseguir y qué aspectos necesitaban corregirse.

Este proceso me ayudó a avanzar en un proyecto con diferentes componentes y a reconocer que programar no consiste únicamente en escribir instrucciones, sino también en probarlas, revisar los resultados y corregir los problemas que aparecen.

Una de las cosas que debo mejorar es dedicar más tiempo a explicar lo que hace cada parte del código antes de aplicar una solución. De esa manera podré comprobar si realmente entiendo el concepto y no dependo únicamente de una respuesta preparada.

Mi siguiente paso es combinar la práctica con ejercicios individuales. Quiero poder leer un fragmento de código, explicar su funcionamiento, predecir su resultado y proponer una modificación sin necesitar que me den toda la solución.

5. Plan de estudio

1. Clases, objetos y constructores

Crear una clase Cadete con nombre y energía, definir un constructor, crear dos objetos con datos diferentes y explicar la función de cada instrucción.

2. Encapsulamiento

Crear una propiedad Energia que solo permita valores entre cero y cien. Practicar los modificadores de acceso y explicar por qué no siempre conviene permitir la modificación directa de los datos.

3. Herencia y polimorfismo

Crear una clase base Personaje con un método Actuar() y dos clases derivadas que sobrescriban ese método. Ejecutar ambos objetos y explicar el uso de virtual y override.

4. Interfaces

Crear una interfaz IExplorable con el método Explorar() e implementarla en dos clases diferentes. Explicar qué tienen en común y por qué cada una puede realizar la acción de forma distinta.

5. Listas genéricas y LINQ

Crear una lista de cadetes con diferentes niveles de energía. Filtrar los que tengan al menos cincuenta puntos con Where, obtener sus nombres con Select y ordenarlos con OrderBy. Después, comparar el resultado con una solución que utilice ciclos y condiciones.

Estos ejercicios me ayudarán a reforzar la teoría y demostrar que puedo aplicar los conceptos de forma independiente.

6. Conclusión

La misión NEXUS me permitió trabajar con un proyecto de programación que integra diferentes funcionalidades y estructuras de código. Durante el proceso participé en decisiones sobre el sistema, comuniqué errores, busqué soluciones y avancé en la organización de sus componentes.

Mi experiencia práctica me proporciona una base para estudiar los temas del Hito 3, especialmente las clases, los objetos, las listas y algunas operaciones de LINQ que aparecen en el código. Al mismo tiempo, reconozco que necesito practicar más para explicar los constructores, el encapsulamiento, la herencia, el polimorfismo y las interfaces sin depender de ejemplos ya resueltos.

Antes del examen, mi objetivo es poder escribir una clase sencilla, crear objetos, explicar sus propiedades y métodos, reconocer relaciones entre clases, aplicar una interfaz, trabajar con listas y predecir el resultado de un fragmento de código. También debo poder detectar errores y justificar qué cambiaría al modificar una instrucción.

Considero que he avanzado en la parte práctica del proyecto, pero quiero seguir reforzando la parte teórica para tener más seguridad al explicar mis decisiones. Mi meta es comprender lo que estoy programando y poder defenderlo con mis propias palabras.

-